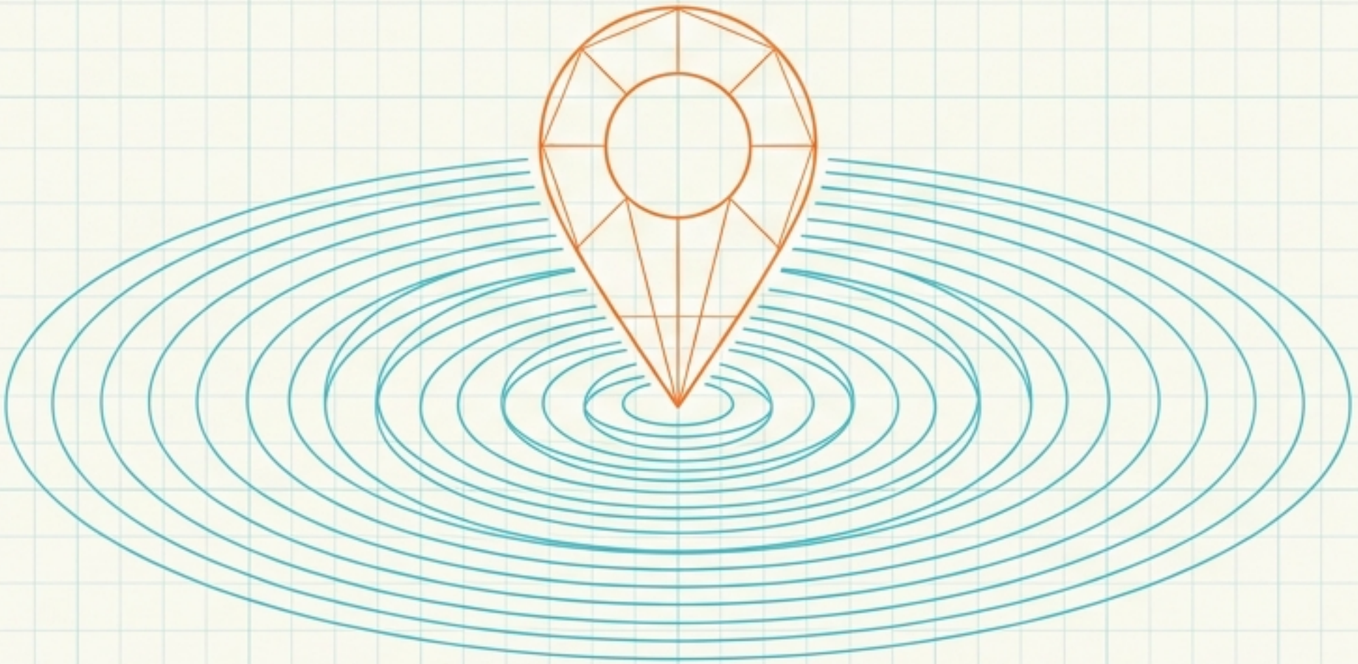


Unit 5: Location Services and Maps

The Android Developer's Guide to Geospatial Architecture

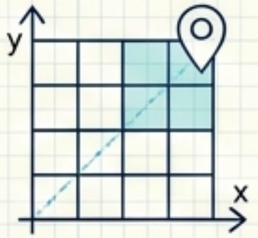
`Acquisition => Visualization => Translation`

The Android Navigation Stack

Step 1: The Sensor Layer (Acquisition)

Topic => Location Services

Function => Extracting raw Latitude/Longitude coordinates from hardware.



Step 2: The Canvas Layer (Visualization)

Topic => Google Maps

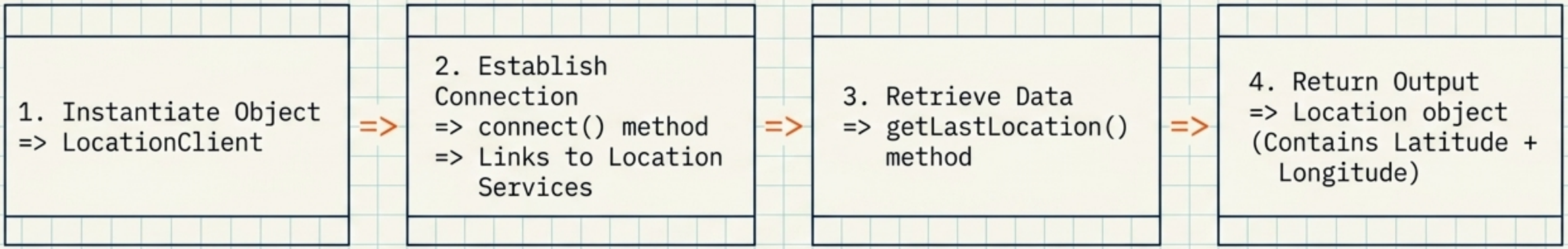
Function => Rendering spatial data onto a digital UI grid.

Step 3: The Translation Layer (Context)

Topic => Geocoding & Reverse Geocoding

Function => Converting raw coordinates into human-readable addresses.

5.1 Getting the Current Location



Requirement => Must implement ConnectionCallbacks and OnConnectionFailedListener interfaces.

Connection Callbacks Lifecycle

Callback Method	Trigger Event	Developer Action
onConnected(Bundle)	Event => Location service successfully connected.	Action => Call connect() to location client.
onDisconnected()	Event => Client is disconnected.	Action => Call disconnect() method.
onConnectionFailed(Result)	Event => Error connecting client to service.	Action => Handle failure/retry.

5.1.2 Location Listener & Providers

LocationListener Interface

Trigger => LocationManager detects location change.

1. onLocationChanged(Location)
   => Triggers on coordinate shift.
2. onProviderDisabled(String)
   => Triggers when user disables provider.
3. onProviderEnabled(String)
   => Triggers when user enables provider.
4. onStatusChanged(String, int, Bundle)
   => Triggers on status shift.

Acquiring Data (The Providers)

GPS_PROVIDER	NETWORK_PROVIDER
Source => Satellites	Source => Cell towers & WiFi access points
Speed => Slower (Takes time for a fix)	Speed => Faster than GPS

5.2 Working with Google Map (Initialization)

Step 1 => Integrate Google Play Services library.

Step 2 => Register application on Google Developer Console.

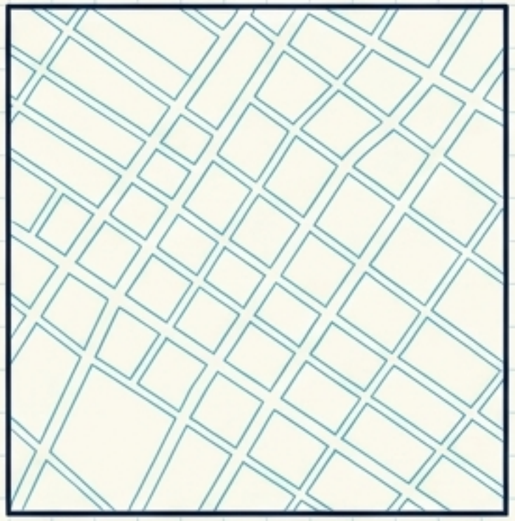
Step 3 => Enable Google Maps API.

Step 4 => Utilize MapActivity for MapView features.

Supplemental Web APIs
- Google Maps JavaScript API - Google Street View Image API - Google Static Maps API - Google Maps Embed API

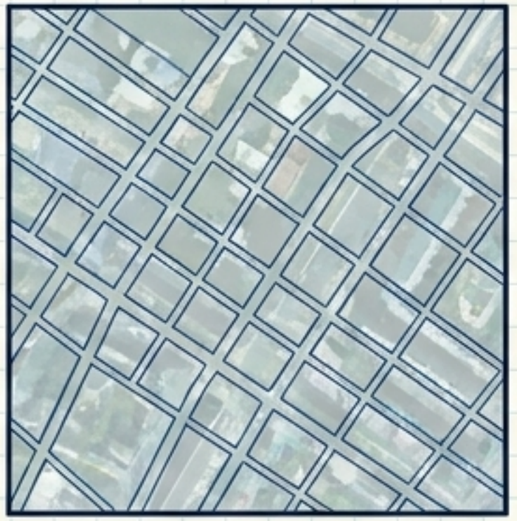
5.2.5 Types of Google Maps

Pre-requisite Flow => GoogleMap.setMapType(constant)

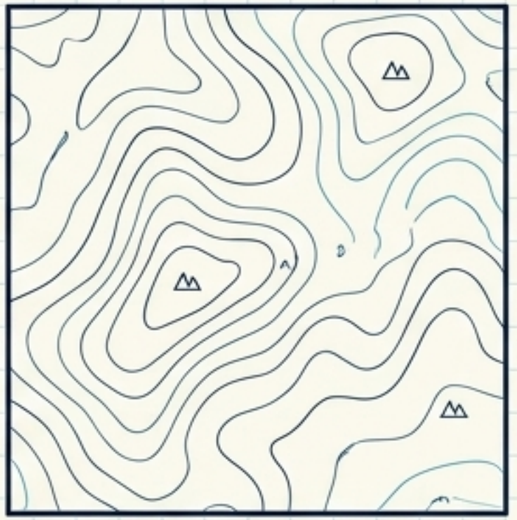


NORMAL

SATELLITE



HYBRID



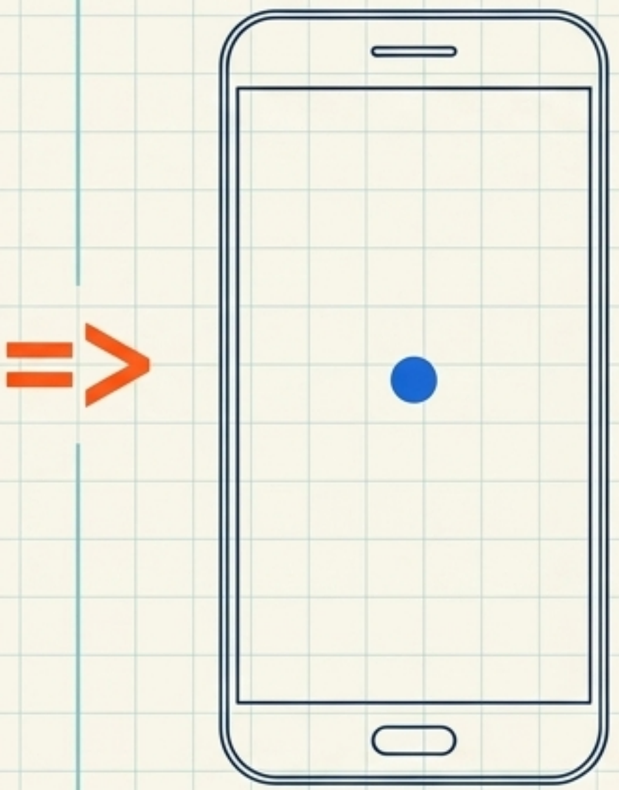
TERRAIN

1. MAP_TYPE_NONE => Empty grid / no mapping tiles.
2. MAP_TYPE_NORMAL => Classic road map (Default).
3. MAP_TYPE_SATELLITE => Google Earth satellite imagery.
4. MAP_TYPE_HYBRID => Satellite imagery + Road map superimposed.
5. MAP_TYPE_TERRAIN => Physical map (contour lines & colors).

5.2.4 & 5.2.6 Implementation & Current Location

XML Implementation

- **UI Layout** => Add to activity_map_demo.xml
- **Element** => <fragment>
- **Class Name** => com.google.android.gms.maps.SupportMapFragment

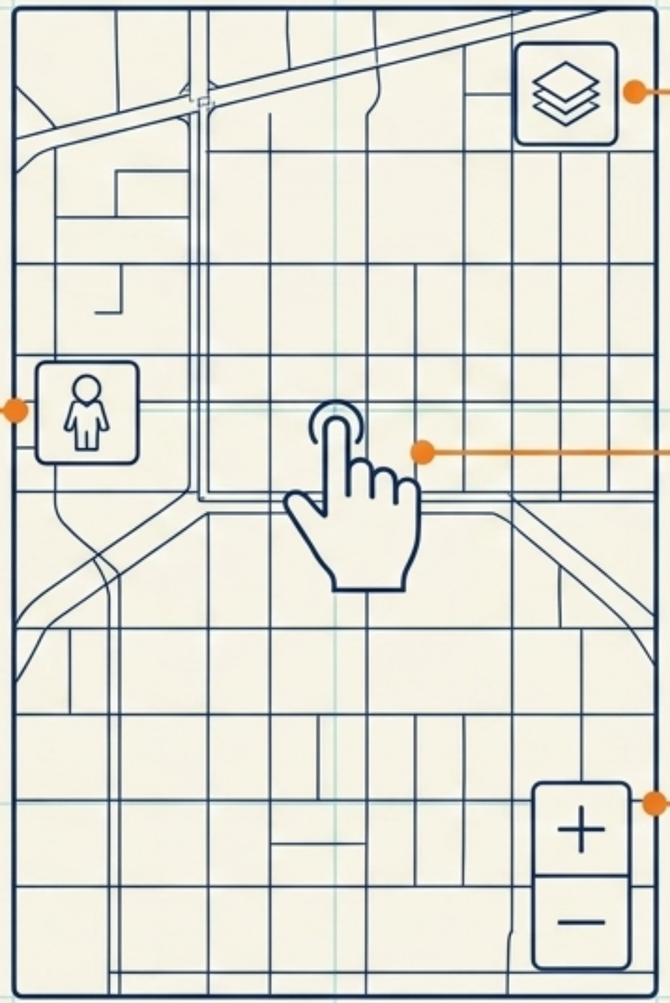


Displaying User Location

- **Prerequisite 1** => onMapReady() is called.
- **Prerequisite 2** => onRequestPermissionsResult() is verified.
- **Action** => GoogleMap.setMyLocationEnabled(true)
- **Result** => Blue dot appears on the rendered canvas.

5.2.7 Anatomy of Map UI Controls

Pre-requisite => Controlled via UiSettings class



Street View (3)

- Location: Draggable icon.
- Function: Opens specific street level views.

(2) Map Type

- Location: Top right.
- Function: Toggles Satellite, Road, Terrain.

(4) Pan Control

- Location: Screen gesture.
- Function: Used for panning the map.

(1) Zoom Controls

- Location: Bottom right.
- Function: Adjust map zoom level.

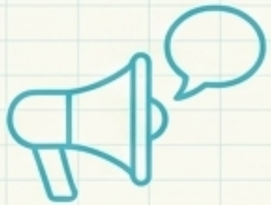
5.2.8 Core Features of Google Maps

1. Route directions & search

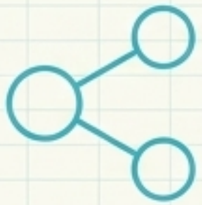
2. Distance information

3. Traffic details & navigation

4. Street views



5. Verbal instructions

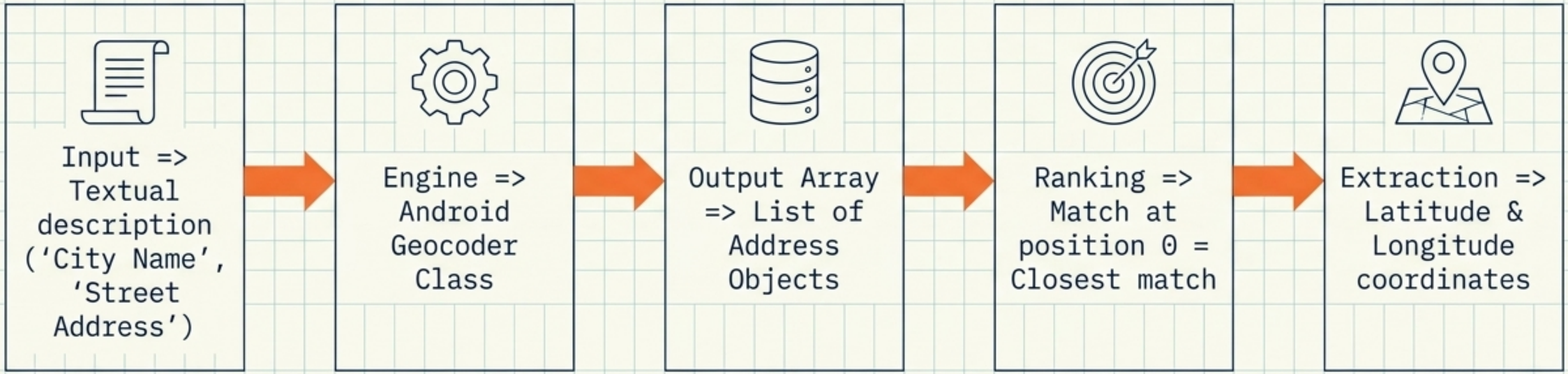


6. Location sharing & editing

5.2.9 Comparative Analysis: Maps vs. Earth

Dimension	Google Maps	Google Earth
Primary Utility	Focus => Navigation	Focus => 3D visual exploration
Architecture	Weight => Lightweight	Weight => Heavy (Full 3D satellite view)
Data Density	Contains => Complete navigation info, hints of satellite	Contains => Small subset of location info
Routing	Capability => Point-to-point navigation	Capability => NO point-to-point navigation

5.3 The Geocoding Pipeline



Process => Transforming descriptive text into geographic coordinates.

5.3.1 Use Cases

- **Uber / Lyft** => Cab booking
- **Google Maps** => Map services
- **Swiggy / Zomato** => Food delivery
- **Fitbit** => Fitness tracking
- **Instagram / Facebook** => Photo tagging

5.3.2 Effectiveness Parameters

Gauge 1: Size of Database

Metric => Fuller base = Higher accuracy.

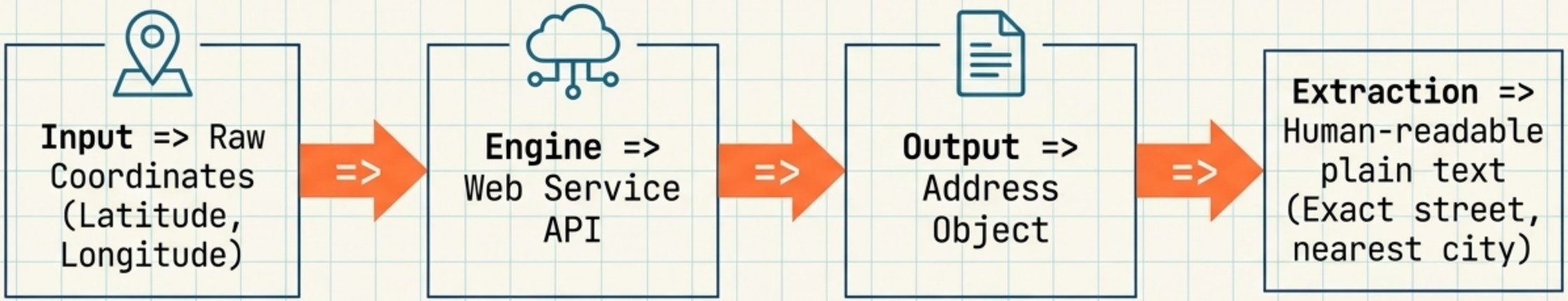
Result => Can specify exact house, even in small towns.

Gauge 2: Geocoding Speed

Metric => Requests processed per second.

Result => Higher speed = System handles more simultaneous users.

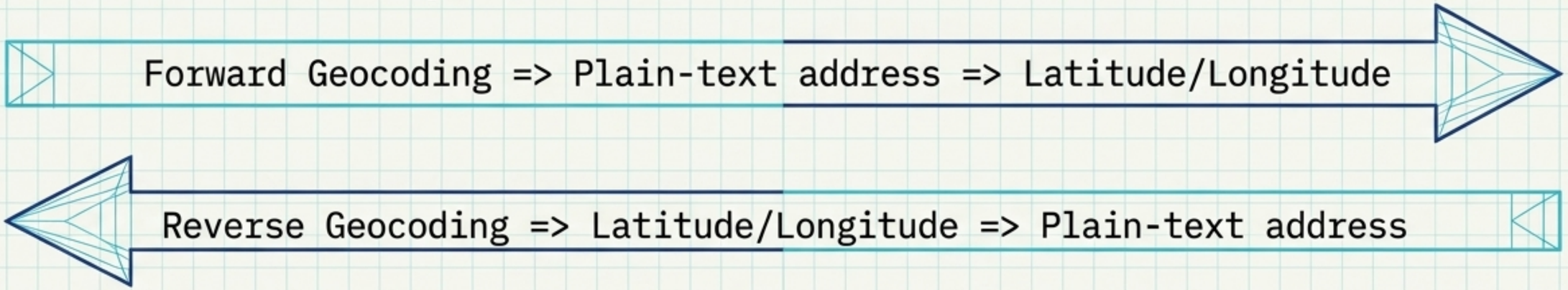
5.4 Reverse Geocoding



Process => Transforming geographic coordinates into a partial or full address description.

Why an API? => Coordinates are static; physical streets change. External API provides reliable mapping.

5.4.1 Geocoding Synthesis



Note: Both return extensive location arrays and confidence scores.

5.5 Isolated Concept: Alarm Manager Service

Function => Triggers specific code at a specific time.
Scope => Runs independently of the application's lifecycle.
Implementation Flow:

1. Get instance from Android SDK system.
2. Pass a PendingIntent.
3. Execution occurs at specified future time.